



Faculdade de Ciências da Universidade de Lisboa

Break-it Phase Vulnerability Analysis

Secure Gallery Log Systems

José Sampaio, fc66191
Rodrigo Neto, fc59850
Gabriel Cambraia, 58671

Contents

1	Introduction	3
2	Evaluation Criteria Used	3
3	Implementation G1: Group 1	3
3.1	Target Summary	3
3.2	How We Built and Ran the Implementation	4
3.3	Reconnaissance and Code Review Methodology	4
3.4	Attack Summary	5
3.5	Finding G1-A1: AES-GCM Nonce Reuse Due to Deterministic IV Generation	5
3.5.1	High-Level Description	5
3.5.2	Root Cause	5
3.5.3	Exploit Preconditions	5
3.5.4	Reproduction Steps	6
3.5.5	Expected Output on a Correct Implementation	6
3.5.6	Observed Output on Group 1	6
3.5.7	Security Impact	6
3.5.8	Attack Code	7
3.6	Finding G1-A2: Potential Integrity Forgery via AES-GCM Nonce Reuse .	7
3.6.1	Description	7
3.7	Vulnerabilities Considered but Not Exploited	7
3.8	Conclusion for Group 1	7
4	Implementation G21: Group 21	7
4.1	Target Summary	7
4.2	How We Built and Ran the Implementation	8
4.3	Reconnaissance and Code Review Methodology	8
4.4	Attack Summary	9
4.5	Finding G21-A1: Weak PBKDF2 Parameters Enable Offline Token Recovery	9
4.5.1	High-Level Description	9
4.5.2	Root Cause	9
4.5.3	Exploit Preconditions	9
4.5.4	Reproduction Steps	9
4.5.5	Expected Output on a Correct Implementation	10
4.5.6	Observed Output on Group 21	10
4.5.7	Security Impact	10
4.5.8	Attack Code	10
4.6	Finding G21-A2: Missing File Locking Allows Concurrent Integrity Violation	10
4.6.1	High-Level Description	10
4.6.2	Root Cause	11
4.6.3	Exploit Preconditions	11
4.6.4	Reproduction Steps	11
4.6.5	Expected Output on a Correct Implementation	11
4.6.6	Observed Output on Group 21	11
4.6.7	Security Impact	12
4.6.8	Attack Code	12

4.7	Finding G21: Additional Vulnerability or Negative Result	12
4.7.1	Description	12
4.8	Vulnerabilities Considered but Not Exploited	12
4.9	Conclusion for Group 21	12
5	Submitted Attack Artifacts	12
6	Overall Conclusion	13

1 Introduction

This document reports our vulnerability analysis for the Break-it phase of the SRSD Gallery Log project. We were assigned two independent implementations to examine: Group 1 and Group 21. For each implementation, we document how we built and ran the submitted programs, what parts of the security design we reviewed, which attacks we attempted, and whether we found a reproducible security vulnerability.

According to the project statement, valid Break-it findings include crashes, integrity violations, and confidentiality violations. We therefore distinguish security-relevant vulnerabilities from ordinary correctness bugs. For each successful attack, we provide a high-level summary, the root cause, reproduction steps, the expected behavior of a correct implementation, the observed vulnerable behavior, and references to any attack code or testcase files submitted alongside this report.

2 Evaluation Criteria Used

We evaluated each assigned implementation against the following security properties from the project specification:

- **Crash resistance:** malformed or adversarial log files should not cause abnormal termination; they should be rejected cleanly.
- **Integrity:** an attacker without the token should not be able to modify, delete, insert, reorder, or forge log entries so that `logread` accepts the resulting file.
- **Confidentiality:** an attacker without the token should not be able to infer private log contents or query results from the file representation alone.
- **Authentication:** commands using the wrong token should not be able to append to or read an existing log.
- **State consistency:** invalid gallery transitions should be rejected and should not leave the log in a partially modified state.

The attack templates below follow the testcase format requested in the project statement: create or obtain a valid log, perform the adversarial step without the token, and then show how the target implementation processes the resulting file incorrectly.

3 Implementation G1: Group 1

3.1 Target Summary

- **Assigned implementation:** Group 1.
- **Language and build system:** Java 17 with Maven.
- **Programs analyzed:** `logappend` and `logread`.
- **Analysis status:** Successful vulnerability identified and reproduced.
- **Main result:** The implementation reuses the same AES-GCM IV for all records in a log file, causing a catastrophic confidentiality failure and enabling cryptographic attacks against the encrypted log structure.

3.2 How We Built and Ran the Implementation

```
cd SRSD_GalleryLog
mvn package

./logappend -T 1 -K segredo -A -E Alice galeria.log
./logappend -T 2 -K segredo -A -E Alice -R 5 galeria.log

./logread -K segredo -S galeria.log
```

Observed result: The implementation compiled successfully using Maven and generated two executable wrapper scripts, `logappend` and `logread`, which invoke the packaged JAR files. Basic functionality tests succeeded, including append operations, room transitions, history queries, integrity verification, and batch mode execution.

3.3 Reconnaissance and Code Review Methodology

We reviewed the implementation with the security goals from the project statement in mind: confidentiality of the log contents without the token, integrity against unauthorized modification, authenticity of appends and reads, and correct handling of invalid gallery-state transitions.

- **Log format:** Each log file begins with a fixed header containing a magic value (BLOG), a version byte, and a random salt. Each appended record stores:

`length || IV || AES-GCM(ciphertext)`

The plaintext encrypted inside each record contains:

`prevHash || timestamp || payload`

where `prevHash` is the SHA-256 hash of the previous raw record.

- **Key derivation and authentication:** The implementation derives a 256-bit AES key from the provided token using PBKDF2-HMAC-SHA256 with 120000 iterations and a per-log random salt. AES-GCM authentication tags are used to verify ciphertext integrity.
- **Encryption design:** The implementation uses AES-256-GCM for authenticated encryption. However, the IV generation is critically flawed. The IV is deterministically generated from the log filename using:

```
SecureRandom vulnerableRandom = new SecureRandom();
vulnerableRandom.setSeed(logPath.getBytes(...));
vulnerableRandom.nextBytes(iv);
```

As a result, every record appended to the same log file reuses the exact same AES-GCM IV under the same encryption key. AES-GCM nonce reuse is cryptographically catastrophic and breaks confidentiality guarantees.

- **Integrity checks:** The implementation validates a SHA-256 hash chain linking consecutive records. Any modification, deletion, truncation, or byte tampering inside a record causes AES-GCM authentication failure or hash-chain mismatch detection.
- **Input validation:** The implementation validates timestamps, room identifiers, state transitions, duplicate flags, malformed arguments, and ordering constraints using a dedicated state machine.

3.4 Attack Summary

ID	Type	Status	Impact
G1-A1	Confidentiality	Exploited	AES-GCM nonce reuse leaks plaintext relationships
G1-A2	Integrity	Not fully exploited	Potential forgery conditions due to nonce reuse

Table 1: Summary of findings for Group 1

3.5 Finding G1-A1: AES-GCM Nonce Reuse Due to Deterministic IV Generation

3.5.1 High-Level Description

The implementation deterministically derives the AES-GCM IV from the log filename instead of generating a fresh random nonce for every encrypted record. Consequently, all records stored in the same log file reuse the same IV under the same AES key.

AES-GCM requires nonce uniqueness for security. Reusing a nonce with the same key completely breaks confidentiality and may enable forgery attacks. An attacker without the token can compare ciphertexts from different records and derive relationships between plaintext contents.

3.5.2 Root Cause

The vulnerability originates in the `CryptoService.randomIv()` method:

```
static byte[] randomIv(String logPath) {
    byte[] iv = new byte[Constants.GCM_IV_BYTES];
    SecureRandom vulnerableRandom = new SecureRandom();
    vulnerableRandom.setSeed(logPath.getBytes(StandardCharsets.UTF_8));
    vulnerableRandom.nextBytes(iv);
    return iv;
}
```

Because the seed depends only on the log path, every append operation targeting the same file generates the same IV. The resulting AES-GCM nonce reuse violates the security assumptions of GCM mode.

3.5.3 Exploit Preconditions

- The attacker can access the encrypted log file but does not know the token.

- The attacker can observe multiple encrypted records stored in the same log.
- The attacker does not need to modify the file or interact with the legitimate user.

3.5.4 Reproduction Steps

```
# 1. Create a log with multiple records
rm -f nonceReuse.log

./logappend -T 1 -K secret -A -E Alice nonceReuse.log
./logappend -T 2 -K secret -A -G Bob nonceReuse.log
./logappend -T 3 -K secret -A -E Alice -R 1 nonceReuse.log

# 2. Observe the binary log contents
xxd nonceReuse.log

# 3. Compare encrypted records
# The IV prefix of every encrypted record is identical.
```

3.5.5 Expected Output on a Correct Implementation

Each encrypted record should use a unique random IV.

3.5.6 Observed Output on Group 1

The IV bytes at the beginning of each encrypted record are identical across all records stored in the same log file.

3.5.7 Security Impact

The attack is security-relevant because AES-GCM nonce reuse completely invalidates the confidentiality guarantees of the encrypted log. Since multiple plaintexts are encrypted using the same keystream, an attacker can compute XOR relationships between plaintext records without knowing the encryption key.

The plaintext structure is highly predictable because every record contains:

- a fixed-size previous hash field,
- a timestamp,
- structured payload fields,
- repeated names and room identifiers.

This predictability enables statistical plaintext recovery and cryptographic analysis of encrypted records. Under certain conditions, AES-GCM nonce reuse may also enable message forgery attacks.

3.5.8 Attack Code

The attack requires no token knowledge and no custom exploit code. The vulnerability can be demonstrated directly by examining multiple encrypted records inside the same log file using:

```
xxd nonceReuse.log
```

3.6 Finding G1-A2: Potential Integrity Forgery via AES-GCM Nonce Reuse

3.6.1 Description

During the analysis, we identified that the AES-GCM nonce reuse vulnerability may also enable integrity attacks or ciphertext forgery attempts. Although we did not complete a practical forgery exploit during the Break-it phase, the cryptographic misuse significantly weakens the authenticated encryption guarantees of AES-GCM.

Because the implementation correctly validates hash chains and authentication tags, we were unable to produce a complete unauthorized append accepted by `logread`. Nevertheless, nonce reuse under AES-GCM is considered a severe cryptographic failure.

3.7 Vulnerabilities Considered but Not Exploited

- **Record reordering attack:** We tested whether records could be reordered while preserving the hash chain. The implementation correctly rejected reordered logs due to previous-record hash validation.
- **Malformed length fields:** We investigated whether corrupted record length fields could trigger parser crashes or uncontrolled memory allocation. The implementation included explicit upper bounds on record size and rejected malformed lengths safely.

3.8 Conclusion for Group 1

The strongest finding against Group 1 was a catastrophic AES-GCM nonce reuse vulnerability caused by deterministic IV generation based solely on the log filename. Although the implementation correctly enforced authentication, state validation, and hash-chain integrity, the cryptographic misuse fundamentally broke confidentiality guarantees for encrypted records.

Overall, the implementation demonstrated good defensive programming practices in several areas, but the IV generation flaw constitutes a severe cryptographic design error.

4 Implementation G21: Group 21

4.1 Target Summary

- **Assigned implementation:** Group 21.
- **Language and build system:** C with Makefile and OpenSSL.
- **Programs analyzed:** `logappend` and `logread`.

- **Analysis status:** Successful attacks found.
- **Main result:** We identified multiple security-relevant vulnerabilities, including a practical confidentiality attack caused by weak PBKDF2 key derivation parameters and an integrity violation caused by missing synchronization during concurrent appends.

4.2 How We Built and Ran the Implementation

Observed result: The implementation compiled successfully using GCC and OpenSSL on Ubuntu Linux. Both `logappend` and `logread` executed correctly during normal operation. No special setup beyond OpenSSL development libraries was required.

```
# Build commands
cd group21
make

# Basic smoke test
./logappend -T 1 -K testToken -E Alice -A test.log
./logread -K testToken -S test.log
```

4.3 Reconnaissance and Code Review Methodology

We reviewed the implementation with emphasis on cryptographic design, state consistency, log integrity enforcement, and malformed-input handling.

- **Log format:** The log file consists of a plaintext salt header followed by an encrypted metadata header and a sequence of encrypted append-only entries. Each entry stores metadata including a sequence number, previous authentication tag, plaintext length, IV, ciphertext, and AES-GCM authentication tag.
- **Key derivation and authentication:** Authentication tokens are converted into AES-256 keys using PBKDF2-HMAC-SHA256. However, the implementation uses only one PBKDF2 iteration, which is far below accepted security standards and enables efficient offline brute-force attacks.
- **Encryption design:** Entries are encrypted using AES-256-GCM with random 12-byte IVs. Additional authenticated data (AAD) includes the sequence number and previous tag to provide chained integrity protection.
- **Integrity checks:** The implementation validates sequence ordering and previous-tag chaining during log loading. However, synchronization between concurrent writers is absent, allowing integrity violations under race conditions.
- **Input validation:** The implementation validates names, timestamps, room identifiers, and gallery-state transitions. However, file locking and crash consistency protections are missing.

4.4 Attack Summary

ID	Type	Status	Impact
G21-A1	Confidentiality	Exploited	Offline brute-force recovery of token and log contents
G21-A2	Integrity	Exploited	Concurrent appends allow inconsistent gallery state

Table 2: Summary of findings for Group 21

4.5 Finding G21-A1: Weak PBKDF2 Parameters Enable Offline Token Recovery

4.5.1 High-Level Description

This implementation derives AES encryption keys using PBKDF2-HMAC-SHA256 with only a single iteration. This allows an attacker with access to the log file to perform an efficient offline brute-force attack against the authentication token. Once the token is recovered, the attacker can decrypt all log contents and forge arbitrary valid log entries.

This is a confidentiality violation because protected log contents become recoverable without authorization.

4.5.2 Root Cause

The vulnerability is caused by the following key derivation code:

```
PKCS5_PBKDF2_HMAC(token, strlen(token),
                  salt, 16,
                  1, EVP_sha256(),
                  32, key)
```

The PBKDF2 iteration count is set to 1. PBKDF2 is intended to slow down brute-force attacks by performing many repeated hashing operations. Using only one iteration makes key derivation computationally trivial and enables rapid password guessing.

4.5.3 Exploit Preconditions

- The attacker can read the encrypted log file.
- The attacker does not know the authentication token.
- The attacker can perform offline computation against the file contents.
- The attacker does not require execution access to the target system.

4.5.4 Reproduction Steps

The following testcase demonstrates the issue.

```
# 1. Create a valid encrypted log
rm -f testcase_group21_A1.log

./logappend -T 1 -K weakpassword -E Alice -A testcase_group21_A1.log
```

```
# 2. Run offline brute-force script
python3 attack_group21_A1.py testcase_group21_A1.log

# 3. Observe recovered token
```

4.5.5 Expected Output on a Correct Implementation

A secure implementation should make offline brute-force attacks computationally infeasible by using a strong password hashing configuration such as Argon2id or PBKDF2 with hundreds of thousands of iterations.

4.5.6 Observed Output on Group 21

```
Recovered token: weakpassword
Successfully decrypted log header.
```

4.5.7 Security Impact

An attacker who obtains the log file can recover weak or medium-strength authentication tokens efficiently. After recovering the token, the attacker can decrypt private gallery logs and forge valid encrypted entries indistinguishable from legitimate ones.

This violates the confidentiality and authenticity goals of the project specification.

4.5.8 Attack Code

File submitted:

```
attacks/group21/attack_group21_A1.py
```

Run using:

```
python3 attack_group21_A1.py testcase_group21_A1.log
```

4.6 Finding G21-A2: Missing File Locking Allows Concurrent Integrity Violation

4.6.1 High-Level Description

The implementation does not use file locking or synchronization when appending entries. Two simultaneous `logappend` processes can therefore validate the same previous state concurrently and both append conflicting transitions successfully.

This is an integrity violation because the resulting log contains states that should have been rejected by the gallery state machine.

4.6.2 Root Cause

The implementation performs validation and appending in separate non-atomic operations:

1. Load current state
2. Validate requested transition
3. Append encrypted entry

No file locking mechanism such as `flock()` or transactional append protocol is used. Multiple processes therefore race against each other using stale state.

4.6.3 Exploit Preconditions

- The attacker can execute multiple `logappend` commands concurrently.
- The attacker knows the correct authentication token.
- Both commands target the same log file simultaneously.

4.6.4 Reproduction Steps

The following testcase demonstrates the issue.

```
rm -f testcase_group21_A2.log

./logappend -T 1 -K raceToken -E Alice -A testcase_group21_A2.log

# Simultaneous conflicting appends
(
./logappend -T 2 -K raceToken -E Alice -A -R 10 testcase_group21_A2.log &
./logappend -T 3 -K raceToken -E Alice -A -R 20 testcase_group21_A2.log &
wait
)

# Read resulting state
./logread -K raceToken -S testcase_group21_A2.log
```

4.6.5 Expected Output on a Correct Implementation

One of the concurrent transitions should fail because Alice cannot enter two rooms simultaneously.

4.6.6 Observed Output on Group 21

Both append operations succeed, resulting in inconsistent gallery state depending on append ordering.

4.6.7 Security Impact

The audit log can no longer be trusted as an accurate representation of gallery activity. Concurrent operations allow logically impossible transitions to be accepted, violating the integrity guarantees required by the project specification.

4.6.8 Attack Code

No external attack script was required. The race condition can be reproduced directly using concurrent shell execution as shown above.

4.7 Finding G21: Additional Vulnerability or Negative Result

4.7.1 Description

We additionally identified a crash-consistency issue in batch mode. During batch execution, encrypted entries are appended immediately while the encrypted header is updated only at batch completion. If the process terminates unexpectedly before header update, the file becomes permanently inconsistent and future operations report an integrity violation.

Although this issue primarily causes denial of service rather than unauthorized log modification, it demonstrates incomplete transactional protection.

4.8 Vulnerabilities Considered but Not Exploited

- **Path traversal through logfile names:** The implementation allows relative paths containing multiple directory separators. However, exploitation impact depended on execution privileges and deployment environment, so we did not submit it as a primary attack.
- **Sequence number wraparound:** Entry sequence numbers are truncated from 64-bit to 32-bit integers. While theoretically exploitable after approximately 2^{32} entries, this was not practical within project constraints.

4.9 Conclusion for Group 21

The Group 21 implementation demonstrated strong use of authenticated encryption and chained integrity validation, but several important security weaknesses remained.

The most severe issue was the use of PBKDF2 with only one iteration, enabling practical offline brute-force attacks against authentication tokens. We also demonstrated an integrity violation caused by missing synchronization during concurrent appends.

Overall, while the implementation attempted to provide confidentiality and integrity guarantees, weak key derivation and lack of concurrency protection significantly reduced its effective security.

5 Submitted Attack Artifacts

The following files should be submitted together with this report if attacks are implemented:

- attacks/group1/: [Scripts, modified logs, and README/testcase commands for Group 1.]
- attacks/group21/: [Scripts, modified logs, and README/testcase commands for Group 21.]

6 Overall Conclusion

[Summarize the final results across both assigned implementations. State clearly which attacks were successful, which vulnerabilities were only theoretical or unexploited, and which implementation appeared stronger under the performed analysis.]